Software Requirements Specification

IndustryApp – post-accident repair offer platform · PA1

Authors: Ömer Efe Öztürk, Bahadır İhsan Herdem · **Course:** BIL 482 Design Patterns · **Version:** 1.0 · **Date:** 05/10/2026

Table of Contents

1. Document Task Matrix
2. Introduction
3. Product Perspective
4. Product Functions
5. User Characteristics
6. Constraints
7. System Features (Use Case Based)
    a. Use Case Diagram
    b. UC-01: Role-Based Registration and Login
    c. UC-02: Create Accident Report
    d. UC-03: Receive Job Notification and Submit Offer
    e. UC-04: Select Offer and Reveal Contact Information
    f. Supporting Flow YA-01: Mechanic Approval and Catalog Management
8. Functional Requirements
9. Non-Functional Requirements
10. External Interface Requirements

1. Document Task Matrix

#	Task in creating this document	Responsible	Reviewer
T1	Introduction, product perspective and functions	Bahadır İhsan Herdem	Ömer Efe Öztürk
T2	User characteristics and constraints	Ömer Efe Öztürk	Bahadır İhsan Herdem
T3	UML use case diagram	Bahadır İhsan Herdem	Ömer Efe Öztürk

#	Task in creating this document	Responsible	Reviewer
T4	UC-01 and UC-04 descriptions	Ömer Efe Öztürk	Bahadır İhsan Herdem
T5	UC-02 and UC-03 descriptions	Bahadır İhsan Herdem	Ömer Efe Öztürk
T6	Supporting flow YA-01 and functional requirements	Ömer Efe Öztürk	Bahadır İhsan Herdem
T7	Non-functional requirements	Ömer Efe Öztürk	Bahadır İhsan Herdem
T8	External interface requirements	Bahadır İhsan Herdem	Ömer Efe Öztürk
T9	Final review, formatting, GitHub upload	Bahadır İhsan Herdem	Ömer Efe Öztürk

2. Introduction

This document specifies the requirements of IndustryApp, a web platform that connects vehicle owners after an accident with repair mechanics through sealed offers. It describes four core use cases (UC-01 … UC-04), one supporting admin flow (YA-01), the functional requirements derived from them (FR-0n.m) and the non-functional requirements. The same numbers are used in the design document, the code comments, the commits and the tests.

Definitions: *Customer* = vehicle owner who reports damage. *Mechanic* = repair workshop that receives jobs and submits offers. *Admin* = platform operator. *Report* = accident report with photos, damage items, vehicle type and location. *Offer* = a mechanic's sealed price proposal with line items. *Match* = a mechanic selected by the system to receive a report. *Sealed offer* = an offer visible only to its author and the report owner. *KVKK* = Turkish Personal Data Protection Law.

3. Product Perspective

IndustryApp is a new, stand-alone web application developed as the term project of BIL 482 Design Patterns, and it is meant to continue as a real product. It is not part of a larger system. It consists of a React single-page application running in the browser, a .NET 8 Web API, a PostgreSQL database and a file store, deployed as three Docker containers behind

a reverse proxy. It depends on one external service, an SMTP server for e-mail. The API is designed API-first, so a future mobile client can use the same endpoints and real-time hub.

4. Product Functions

1. Register as a customer or a mechanic with role-specific forms and legal consent; log in and reach a role-specific dashboard.
2. Approve or reject new mechanic accounts (admin).
3. Create an accident report through a five-step wizard with processed photos, damage items from a category tree, vehicle type and location.
4. Automatically match suitable mechanics to a new report and notify them in real time and by e-mail.
5. Submit, update and withdraw a sealed offer with a price breakdown (mechanic).
6. Show offers live to the customer, sort them, and select one.
7. Reveal contact information only to the customer and the selected mechanic; allow undo of the selection within a time window.
8. Mark a report completed or cancel it; expire reports with no selection after a set time.
9. Deliver in-app and e-mail notifications and keep a notification history.
10. Manage the damage category catalog (admin).

5. User Characteristics

- **Customer:** an individual vehicle owner, often under stress after an accident. Uses a phone or computer browser daily but has no technical knowledge. Needs a guided, step-by-step flow, clear status labels and no jargon.
- **Mechanic:** owner or employee of a repair workshop. Digital skills range from low to moderate; often checks jobs on a phone between tasks. Needs instant notifications, short forms and a clear view of their own offers.
- **Admin:** a member of the operating team with good computer skills and knowledge of the trade. Uses a desktop browser to review mechanic applications and maintain the damage catalog.

6. Constraints

- **Technology:** backend .NET 8 / ASP.NET Core Web API (C#); frontend React 18 + TypeScript (Vite); database PostgreSQL 16; real-time SignalR; deployment Docker Compose + Caddy on one Linux server (2 vCPU, 4 GB RAM).

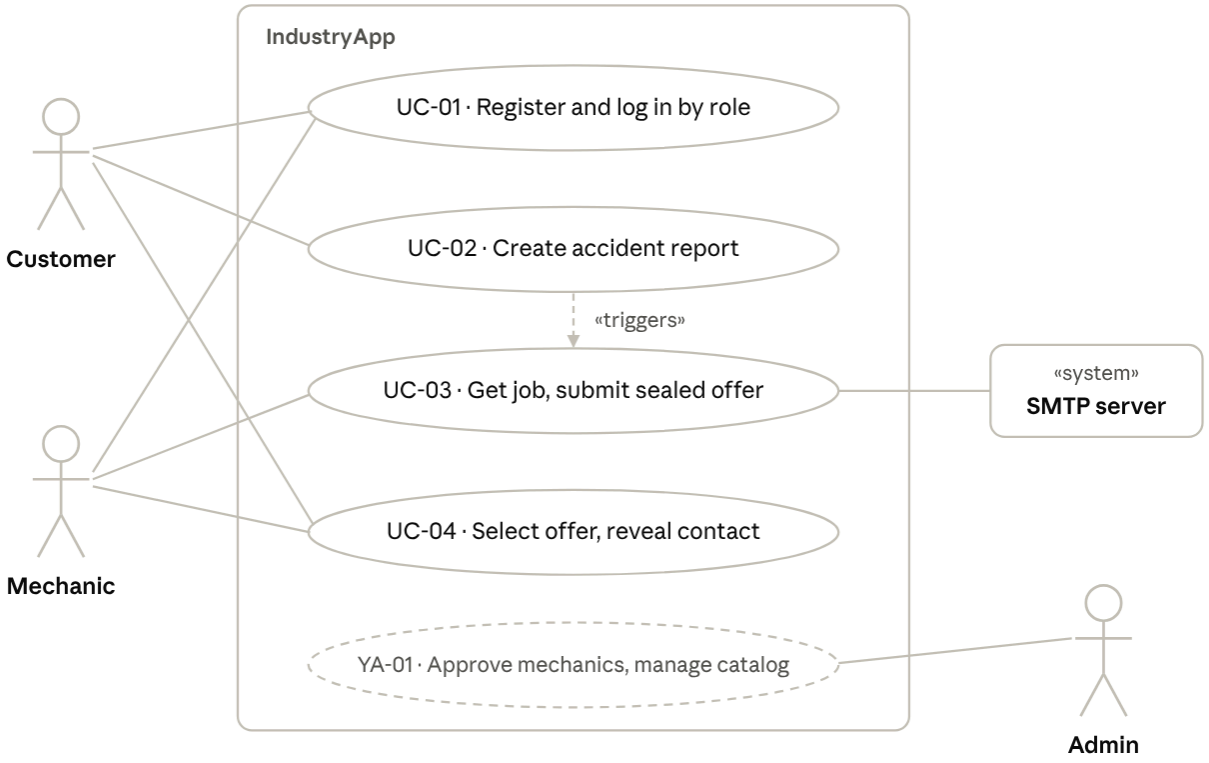
- **Architecture:** Clean Architecture with dependencies pointing inward (WebApi → Application → Domain); modules communicate through events, controllers never call each other.
- **Platform:** web only (current desktop and mobile browsers); no native mobile app in this project.
- **Data:** no seed or demo data; reference data (roles, categories, vehicle types, legal texts) comes from a versioned SQL script, all transaction data from the application's own screens.
- **Legal:** KVKK — consent with document version must be stored at registration; contact details shown only to matched parties; photo location data removed.
- **Configuration:** limits and durations (undo window 10 min, expiry 72 h, 8 photos per report, 10 MB per photo, 3 open reports per customer, 20 mechanics per report) are read from `appsettings.json`, never hard-coded.
- **Team and schedule:** two developers; delivery in three assignments (PA1–PA3) within one semester; each design pattern must be traceable by its UC and PAT number.

7. System Features (Use Case Based)

The system is organized around four core use cases; each is traced by its number to functional requirements, patterns and tests.

7.1 Use Case Diagram

Four core use cases, three human actors, one external system



Dashed ellipse: supporting admin flow, not one of the four core use cases. The mechanic is notified in UC-04.

UML use case diagram · 3 actors, 4 core use cases, 1 supporting flow

The customer takes part in UC-01, UC-02 and UC-04; the mechanic in UC-01, UC-03 and UC-04 (notified). A new report from UC-02 triggers UC-03 automatically through an event, and the SMTP server delivers the e-mails of UC-03.

7.2 UC-01: Role-Based Registration and Login

- **Main actor:** Customer or Mechanic (prospective user) · **Supporting actors:** Admin (approves mechanics, see YA-01)
- **Goal:** the user opens an account with the information their role needs and enters the interface of that role.
- **Preconditions:** the user is not registered; reference data (roles, damage categories, current legal documents) is loaded.

Main flow:

1. The user selects a role on the registration screen (Customer / Mechanic).
2. The system shows the role-specific form and the current legal documents of that role.
3. The user enters the common fields: full name, e-mail, phone, password.

4. A mechanic additionally enters workshop name, tax number, city, district, address, location (optional coordinates from the device) and at least one specialty.
5. The user accepts the legal documents and submits the form.
6. The system creates the profile through the role's profile factory, assigns the role and stores the consents.
7. A customer is logged in and sent to the customer dashboard; a mechanic sees the "waiting for approval" screen.
8. On later logins the system issues a JWT access token and a refresh token and returns the role-specific dashboard data.

Alternative flows and exceptions:

- **3a.** The e-mail is already registered → error `EMAIL_ALREADY_USED`.
- **3b.** Invalid input (e-mail format, phone not `05XXXXXXXXX`, password shorter than 8 characters or without a digit, tax number not 10–11 digits) → field errors, nothing is saved.
- **4a.** The tax number is already registered → error.
- **5a.** A required legal document is not accepted → submission is blocked (`CONSENT_REQUIRED`).
- **8a.** A mechanic who is pending or rejected logs in → sees only the profile status screen; the job list is closed.
- **8b.** Five wrong passwords → the account is locked for 15 minutes.
- **8c.** Too many login or registration attempts from one IP (5 or 3 per minute) → HTTP 429.

Postconditions: the user, role, profile and consent records are created in one transaction; if any step fails, nothing is kept.

Related requirements: FR-01.1 – FR-01.4 · **Patterns:** Factory Method (PAT-01.1), Abstract Factory (PAT-01.2)

7.3 UC-02: Create Accident Report

- **Main actor:** Customer · **Supporting actors:** System (matching, triggered by the result)
- **Goal:** report the damage with photos and structured criteria and open it to mechanics.
- **Preconditions:** the customer is logged in and has fewer than 3 open reports.

Main flow:

1. The customer opens the "New report" wizard.
2. Step 1: uploads 1–8 photos. Each photo is processed on upload (resize, compress, remove EXIF/GPS) and stored temporarily.

3. Step 2: navigates the damage tree and selects one or more damaged areas, each with a severity.
4. Step 3: selects the vehicle type and optionally writes a description (up to 1000 characters).
5. Step 4: takes the location from the device or selects city and district.
6. Step 5: reviews the summary and confirms.
7. The system builds the report with the Builder, rejects it if anything is missing, saves it as Open and sets its expiry time (default 72 hours).
8. After saving, the system publishes `AccidentReportCreatedEvent` and sends the customer to the report detail screen.

Alternative flows and exceptions:

- **2a.** Unsupported file type (only JPEG, PNG, WebP, checked by file signature) or size over 10 MB → that photo is rejected, the others stay.
- **5a.** Location permission is denied → city and district selection becomes mandatory.
- **7a.** The customer already has the maximum number of open reports (default 3) → error `OPEN_REPORT_LIMIT_REACHED`.
- **7b.** A photo key does not belong to this customer, or a category is inactive → the request is rejected.
- **7c.** No photo, no damage item, no location or no vehicle type → `REPORT_INCOMPLETE`.
- **Extension.** If no offer is selected within the expiry time, the report becomes Expired automatically and the customer is notified.

Postconditions: the report, its damage items and photos are saved; photos carry no location metadata; the creation event is queued and the HTTP response does not wait for matching.

Related requirements: FR-02.1 – FR-02.4 · **Patterns:** Builder (PAT-02.1), Composite (PAT-02.2), Decorator – image pipeline (PAT-02.3), Adapter – file storage (PAT-02.4)

7.4 UC-03: Receive Job Notification and Submit Offer

- **Main actor:** Mechanic · **Supporting actors:** System (trigger), Customer (receives the offer), SMTP server
- **Goal:** suitable mechanics learn about a new report immediately and submit a sealed offer.
- **Preconditions:** the mechanic is approved and available; an Open report exists (created in UC-02).

Main flow:

1. The system receives `AccidentReportCreatedEvent`.
2. The active matching strategy (default: by city) finds candidate mechanics.
3. Candidates pass through the filter chain: approved and active → specialty matches → available → daily capacity not full.
4. The remaining mechanics (at most 20) are saved as matches and `MechanicsMatchedEvent` is published.
5. Each mechanic receives an in-app notification (a pop-up if online) and an e-mail.
6. The mechanic opens the notification and sees the report detail. Other mechanics' offers are not visible.
7. The mechanic enters price items (labor, parts, paint, warranty, other) and the estimated number of days.
8. The system computes the total, saves the offer as Pending and publishes `OfferSubmittedEvent`.
9. The offer appears on the customer's screen immediately and a notification is created.

Alternative flows and exceptions:

- **2a.** No candidate is found → the customer is notified that no suitable mechanic is available in the area right now.
- **5a.** The e-mail provider does not respond → the Circuit Breaker opens, the e-mail is recorded as `CircuitOpen`, the in-app notification is still delivered.
- **6a.** A mechanic who is not matched with the report tries to open it or send an offer → 403 `NOT_MATCHED`.
- **7a.** The mechanic tries to send a second offer for the same report → redirected to update the existing offer.
- **7b.** No labor item, or a total of zero → validation error.
- **8a.** The report has meanwhile become OfferSelected → 409, "the report no longer accepts offers".
- **8b.** The mechanic may withdraw a Pending offer or update it while the report is Open.

Postconditions: matches are saved; the offer is stored with its price items and delivered to the customer.

Related requirements: FR-03.1 – FR-03.5 · **Patterns:** Observer / Publish–Subscribe (PAT-03.1), Strategy (PAT-03.2), Chain of Responsibility + Template Method (PAT-03.3), Decorator – price (PAT-03.4), Adapter – notification channels (PAT-03.5), Circuit Breaker + Retry (PAT-03.6)

7.5 UC-04: Select Offer and Reveal Contact Information

- **Main actor:** Customer · **Supporting actors:** Mechanic (notified)

- **Goal:** the customer selects one offer, and only at that moment both parties can see each other's contact details.
- **Preconditions:** the customer owns an Open report with at least one Pending offer.

Main flow:

1. The customer sees the offers on the report detail screen and sorts them by price, estimated duration or newest.
2. The customer opens an offer's price breakdown, presses "Select this offer" and confirms the dialog.
3. The system creates and executes `SelectOfferCommand`: the report becomes OfferSelected, the chosen offer Accepted, the others NotSelected; the previous state is stored as a snapshot in the status history.
4. `OfferSelectedEvent` is published; the winning mechanic is notified "you were selected", the others "another offer was selected".
5. The customer and the winning mechanic see each other's name, phone and e-mail on the match screen; for everyone else the number is masked.
6. During the undo window (default 10 minutes) the customer sees a countdown.
7. When the work is done, the customer marks the report as completed.

Alternative flows and exceptions:

- **2a.** The user is not the owner of the report → 403 `NOT_REPORT_OWNER`.
- **3a.** Two selection requests arrive at the same time → the second fails on the partial unique index or the concurrency token → 409.
- **3b.** The offer was withdrawn in the meantime → 409 `OFFER_NOT_AVAILABLE`.
- **6a.** The customer presses "Undo" within the window → the command is rebuilt from the snapshot and `Undo()` runs; the report returns to Open, all offers to Pending, `OfferSelectionUndoneEvent` is published.
- **6b.** Undo after the window → 409 `UNDO_WINDOW_EXPIRED`.
- **7a.** The customer cancels the report before work starts → the report becomes Cancelled, Pending offers Closed.

Postconditions: the report has exactly one accepted offer; contact details are open only to the two matched parties.

Related requirements: FR-04.1 – FR-04.5 · **Patterns:** State (PAT-04.1), Command (PAT-04.2), Protection Proxy (PAT-04.3), Strategy – sorting (PAT-04.4), Observer (PAT-03.1, reused)

7.6 Supporting Flow YA-01: Mechanic Approval and Catalog Management

YA-01 is not one of the four core use cases, but production use requires it: without approval, mechanics could never receive jobs.

- **Main actor:** Admin · **Precondition:** the admin is logged in (the first admin is assigned by a one-line SQL script, not by public sign-up).
- **Main flow:** the admin lists mechanics waiting for approval, checks tax number and workshop details, and approves or rejects with a reason. The approved mechanic receives an in-app notification and an e-mail and joins matching. The admin also adds damage categories (up to 3 levels, unique code), renames them or deactivates them.
- **Alternative flows:** a rejected mechanic sees the reason on the status screen; a deactivated category disappears from new reports but existing reports are unchanged.

8. Functional Requirements

UC-01 Registration and login

- **FR-01.1** A user registers as customer or mechanic with a role-specific form.
- **FR-01.2** Accepting the role's legal documents is mandatory at registration and is stored with the document version.
- **FR-01.3** A mechanic account cannot receive jobs until an admin approves it.
- **FR-01.4** After login the user is sent to the dashboard of their role.

UC-02 Accident report

- **FR-02.1** The customer uploads 1–8 photos; photos are resized and their location data is removed.
- **FR-02.2** The customer selects one or more damaged areas and a severity from the damage tree.
- **FR-02.3** A report cannot be created with missing information.
- **FR-02.4** A report closes automatically after the configured time if no offer is selected.

UC-03 Job notification and offer

- **FR-03.1** When a report is created, approved and available mechanics matching its city and specialty are found automatically.
- **FR-03.2** Matched mechanics are informed in-app and by e-mail.
- **FR-03.3** A mechanic submits one offer with a price breakdown and can update or withdraw it.
- **FR-03.4** A mechanic cannot see other mechanics' offers.

- **FR-03.5** If the e-mail provider fails, in-app notifications are not affected.

UC-04 Offer selection and contact

- **FR-04.1** The customer sees offers live and sorts them by price, duration or newest.
- **FR-04.2** The customer selects one offer; the other offers are marked not selected.
- **FR-04.3** The customer can undo the selection within the configured time.
- **FR-04.4** Contact details are shown only to the customer and the selected mechanic.
- **FR-04.5** The customer marks the job completed or cancels the report.

9. Non-Functional Requirements

Usability

- **NFR-U1** A first-time customer can create a report without help; the wizard has 5 steps, each validated before the next.
- **NFR-U2** Every screen is in Turkish; codes and enum values are never shown raw but translated to Turkish labels; empty values appear as `-`.
- **NFR-U3** Dates are shown as `dd/MM/yyyy` or `dd/MM/yyyy HH:mm`; prices in Turkish lira format.
- **NFR-U4** The interface is responsive and usable on a phone browser (mechanics often work from a phone).
- **NFR-U5** Every error from the API is shown as a clear Turkish message, never as a technical stack trace.

Performance

- **NFR-P1** API responses stay under 500 ms at the 95th percentile under normal load.
- **NFR-P2** Report creation returns in under 500 ms (p95) independent of matching and notification, which run in the background.
- **NFR-P3** Matched mechanics who are online see the new-job pop-up within 3 seconds of report creation.
- **NFR-P4** New offers and status changes reach the customer's open screen without a page refresh.

Portability

- **NFR-PO1** The client runs in current versions of the major desktop and mobile browsers; nothing needs to be installed.
- **NFR-PO2** The server side runs anywhere Docker runs; the whole system starts with `docker compose up` on a Linux host.

- **NFR-PO3** The API is client-independent (REST + SignalR), so a mobile client can be added without changing the backend.

Reliability

- **NFR-R1** A report can never have two accepted offers; this is enforced by a database constraint, not only by code.
- **NFR-R2** Writes are transactional; events are published only after the transaction commits.
- **NFR-R3** An e-mail provider failure does not stop or slow other notifications (Circuit Breaker with retry and timeout).
- **NFR-R4** After a real-time disconnect the client reconnects automatically and reloads its data; the REST API is the source of truth.
- **NFR-R5** The database and uploaded files are backed up every night and kept for 7 days; a restore is tested before go-live; `/health` reports database connectivity.

Security and privacy

- **NFR-S1** HTTPS only, with HSTS; passwords hashed by ASP.NET Core Identity (minimum 8 characters with a digit; lock for 15 minutes after 5 failures).
- **NFR-S2** Access tokens live 30 minutes, refresh tokens 14 days with rotation; only SHA-256 hashes of refresh tokens are stored.
- **NFR-S3** Role checks on every endpoint and ownership checks in services; contact details only through the protection proxy.
- **NFR-S4** Uploaded files are verified by their signature, renamed with a server GUID and served only through an authorized endpoint.
- **NFR-S5** Rate limits: login 5/min per IP, registration 3/min per IP, upload 30/min per user.
- **NFR-S6** Passwords, tokens and full phone numbers never appear in logs.

Maintainability

- **NFR-M1** Each design pattern is marked with a `[UC-0n][PAT-0n.m]` comment and covered by at least one test.
- **NFR-M2** No duration, limit or address is hard-coded; all come from configuration.
- **NFR-M3** Domain and Application code reach at least 70% test coverage, pattern classes at least 90%.

10. External Interface Requirements

10.1 User Interfaces

The client has 20 screens; after login the user is routed by role (customer → `/customer`, approved mechanic → `/mechanic`, pending mechanic → `/mechanic/pending`, admin → `/admin`).

- **Public (UC-01):** Login, Role selection, Customer registration, Mechanic registration (with "Use my location" and specialty selection), Error pages (403/404).
- **All logged-in users:** Profile (UC-01), Notifications (all UCs), Match screen with the other party's contact details (UC-04), a notification bell in the layout.
- **Customer:** Dashboard (UC-01), My reports (UC-02), New report wizard with 5 steps (UC-02), Report detail with live offer list, sorting, price breakdown, selection, undo countdown, complete and cancel (UC-04).
- **Mechanic:** Dashboard with availability switch (UC-01), Pending approval status (UC-01), Jobs with tabs New / Offered / Won / Lost (UC-03), Job detail with photo gallery and offer form (UC-03), My offers (UC-03).
- **Admin:** Dashboard (YA-01), Mechanic approvals (YA-01), Category management (YA-01).

10.2 Software Interfaces

- **REST API** (`/api/...`, JSON in camelCase): numbered endpoints in thin, resource-based controllers (auth, account, catalog, photos, reports, mechanic jobs, offers, selection, contacts, notifications, admin). Errors use RFC 7807 ProblemDetails with an application `code` and a `traceId`. Authentication with a JWT Bearer token.
- **SignalR hub** (`/hubs/notifications`): server-to-client only, authenticated with the access token; events `notification.created`, `offer.created`, `offer.updated`, `offer.withdrawn`, `report.statusChanged`. All actions go through REST.
- **SMTP server** (external): e-mail delivery through MailKit (port 587, STARTTLS), behind the `INotificationChannel` interface and wrapped by retry + Circuit Breaker. The provider is still to be chosen.
- **SMS channel:** the same `INotificationChannel` interface exists; in this project it only logs a masked message (no provider).
- **PostgreSQL 16:** accessed only through Entity Framework Core 8 (Npgsql) with snake_case naming; schema created by migrations; reference data by `database/reference-data.sql`.
- **File storage:** server disk (Docker volume) behind the `IFileStorage` interface, so S3/MinIO can replace it with one new class.

- **Browser APIs:** Geolocation API for the location step (optional; city/district fallback).

10.3 Communication Interfaces

- HTTPS only through the Caddy reverse proxy (automatic Let's Encrypt certificates); the frontend, `/api` and `/hubs` share one domain, so no cross-origin calls in production.
- WebSocket (with automatic fallback) for SignalR.